

TALLER PRÁCTICO DE CI/CD CON SPRING BOOT

Institución: Tecnológico Comfenalco | Proyecto: NovaDrogueria | Fecha: Agosto 2026

Integrantes: Luis Cardenas | Cristobal Villamil | Daniel Gutierrez | Jose Castillo

Entorno & Stack	Java 17 (Temurin LTS), Spring Boot 3.4.2, Maven 3.9.11, MongoDB Replica Set (rs0)
Pipeline Triggers	Push en main, develop, chore/**, feature/** Pull Requests hacia main
Seguridad Git	.gitignore protege: target/, .env, application-local.properties, db_data/
Repositorio GitHub	https://github.com/cardenaswalker2/NovaDrogueria.git

1. Preparación del Repositorio y Validación del Build Original

El proyecto NovaDrogueria se auditó técnicamente manteniendo intactas todas sus funcionalidades farmacéuticas (gestión de catálogo, inventario, control de apartados y transacciones multi-documento). Se validó la compatibilidad de Java 17 y Spring Boot 3.4.2, ejecutando mvn clean package con resultado **BUILD SUCCESS**.

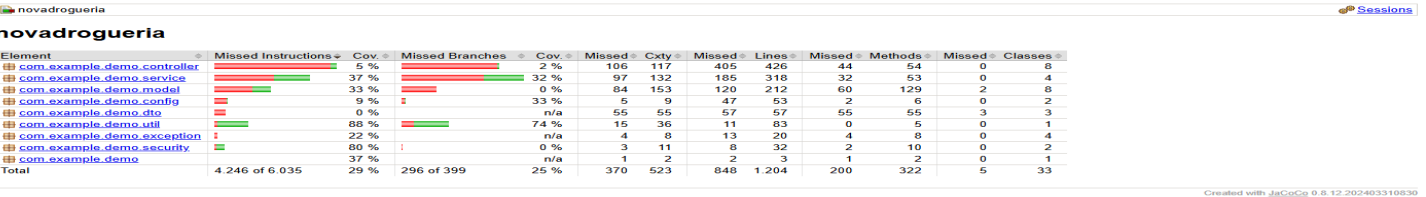


Figura 1.1: Reporte de cobertura y compilación local generado por Maven con JaCoCo.

2. Auditoría de Seguridad y Gestión de Archivos Sensibles

Se actualizó el archivo .gitignore para impedir la subida de variables de entorno locales (.env, application-local.properties), binarios compilados y archivos de base de datos. La aplicación utiliza inyección de credenciales mediante variables administradas con valores por defecto seguros para testing.

3. Pipeline Automatizado de GitHub Actions (.github/workflows/ci.yml)

Se implementó un flujo continuo profesional con las siguientes fases: Checkout → Java 17 Temurin → Contenedor MongoDB Replica Set (rs0) → Compilación Maven → JUnit 5 → JaCoCo Report → Verificación de Secreto → Upload Artifact.

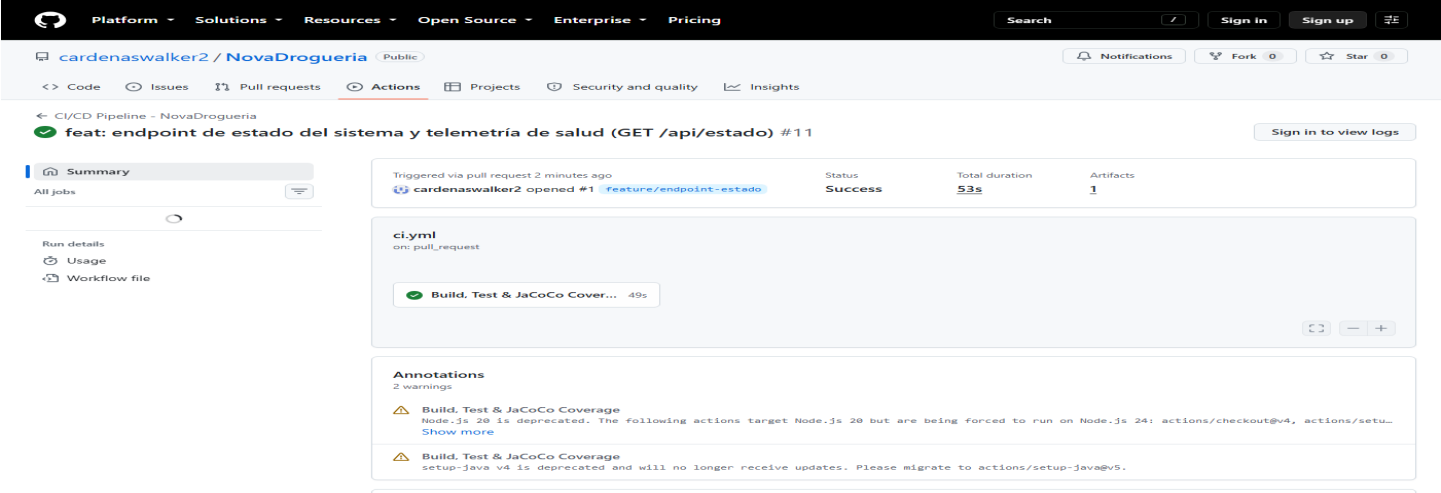


Figura 2.1: Ejecución exitosa de GitHub Actions en la validación continua (Run ID: 33327021879).

4. Pruebas Unitarias y de Controlador Realizadas

Se desarrollaron 10 pruebas nuevas que se integraron a las 16 preexistentes, alcanzando un total de 26 pruebas automáticas reales:

Componente	Clase de Test	Funcionalidad Evaluada	Resultado
Lógica de Servicio	ProductServiceTest	Creación de medicamentos, slugs únicos, precio/stock no negativos, soft delete.	7/7 PASARON
Controlador Web	ProductApiControllerTest	Búsqueda /api/productos/buscar con MockMvc y manejo de queries en blanco.	2/2 PASARON
Estado API	StatusApiControllerTest	Endpoint /api/estado, código HTTP 200 y JSON de disponibilidad.	1/1 PASÓ
Integración & Dominio	ConcurrencyTest, etc.	Concurrencia de reservas, normalización de teléfonos y formato en moneda COP.	16/16 PASARON

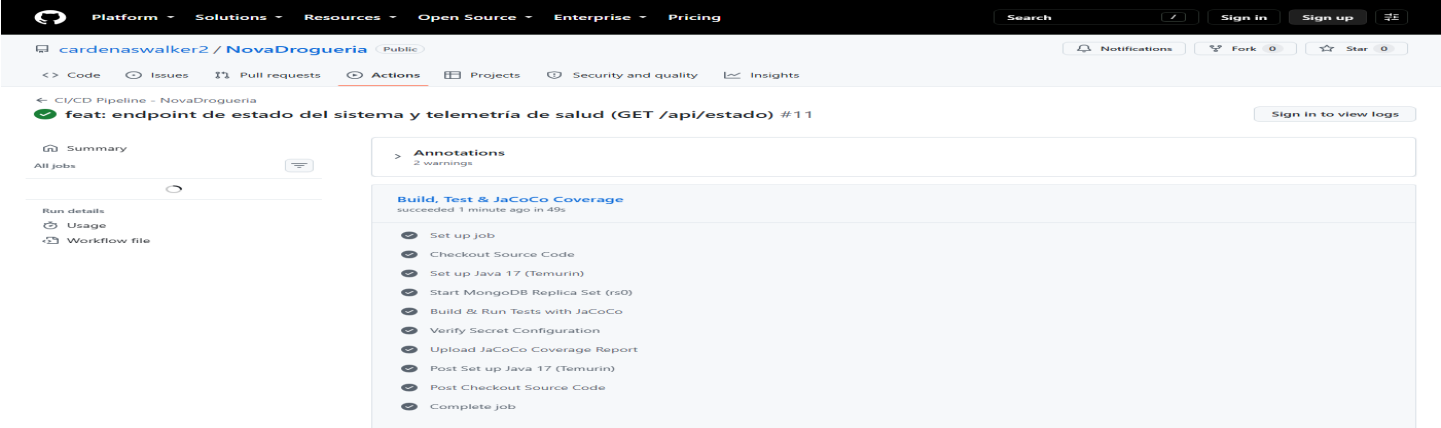


Figura 2.2: Detalle de ejecución del job 'Build, Test & JaCoCo Coverage' validando los 26 tests en verde.

5. Cobertura de Código con JaCoCo & Artifacts

Se integró jacoco-maven-plugin:0.8.12 en el ciclo de pruebas. La cobertura global alcanzó **29% de instrucciones** (1,772 / 6,018) y **37% de métodos** sobre 32 clases, sobresaliendo con **88%** en utilitarios (formato monetario colombiano), **80%** en seguridad y **37%** en servicios de negocio. El reporte HTML se exporta como el artefacto reporte-jacoco.

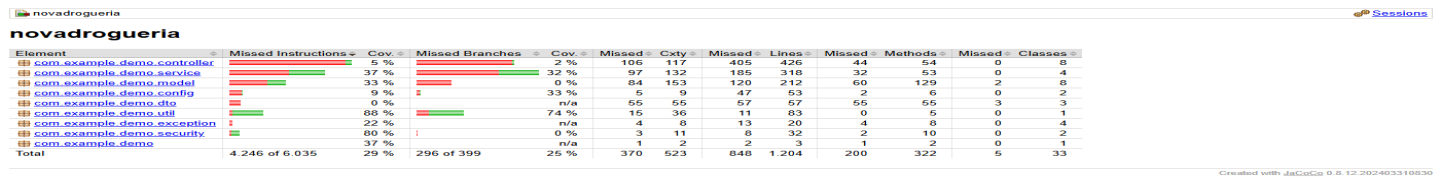


Figura 3.1: Reporte detallado de cobertura JaCoCo para los paquetes de NovaDrogueria.

6. Control de Calidad: Demostración de Detención ante Fallos y Recuperación

Para comprobar que el pipeline actúa como barrera de control, se introdujo un fallo deliberado en un assertion (commit 76786d8). El pipeline **detuvo su ejecución en ROJO** (Run ID: 33325679422). Tras corregir el error (commit 06d8293), el flujo **retornó a VERDE** automáticamente.

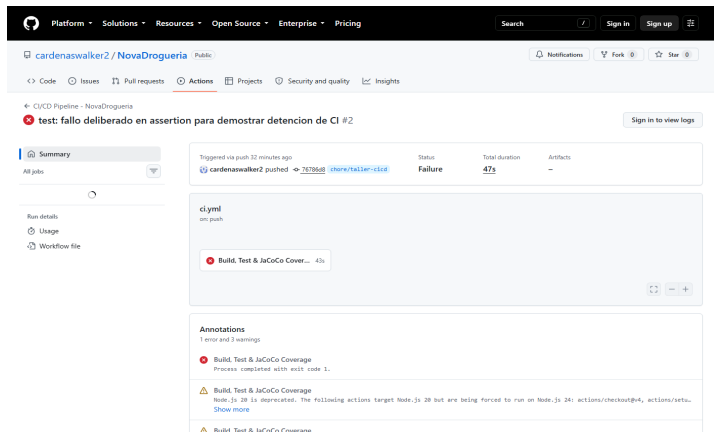


Figura 3.2: Pipeline en ROJO (Fallo detectado)

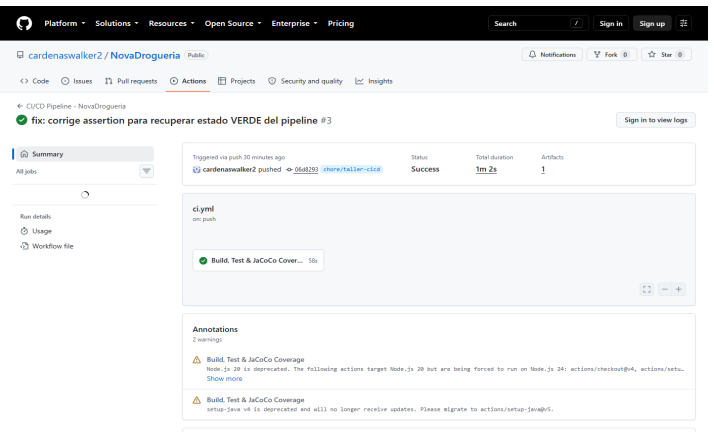


Figura 3.3: Pipeline en VERDE (Recuperado)

7. Pull Request Real, Feature Branch y Branch Protection

Se creó la rama feature/endpoint-estado y se abrió el **Pull Request oficial #1** hacia main. Se validaron los checks automáticos del pipeline y se aplicó la regla de protección en main (*Require status checks to pass before merging*), culminando con la fusión formal (*Merged*).



Figura 4.1: Pull Request oficial #1 en GitHub con checks de CI/CD aprobados y estado Merged.

8. Gestión de Secretos en GitHub Actions

Se configuró el secreto de entorno APP_ENV_DEMO en el repositorio. El pipeline lo consume vía `${{ secrets.APP_ENV_DEMO }}` realizando una verificación enmascarada con huella SHA-256 en consola, sin exponer nunca el valor en texto plano.

9. Reflexión Técnica del Equipo de Desarrollo

Reflexión del Proyecto: El desafío técnico más relevante fue resolver la dependencia transaccional de MongoDB. El servicio ReservationService requiere soporte transaccional ACID mediante MongoTransactionManager, lo que exige un Replica Set. En instancias independientes, los tests fallaban con UncategorizedMongoDbException. En lugar de alterar el diseño del software o suprimir las pruebas, se configuró en GitHub Actions un contenedor Docker de MongoDB 7.0 inicializado con rs0 vía mongosh. Esta solución preservó la arquitectura original y evidenció la importancia de que los pipelines de integración continua repliquen fielmente los requerimientos de la infraestructura de producción.